ILC Lighting App

Session orientation for Manuel San Ramon · July 2026

TO CLAUDE CODE: You have been given this PDF to orient yourself for Manny's session. Read everything on pages 1–2, then the task specs that follow. Greet Manny by name, confirm you understand the project by mentioning a specific detail (e.g. the auth fix in Task 1, or the weighted pipeline formula in Task 3). Tell him Task 1 is four quick backend fixes (~45 min) and is the warmup. Warm tone, short sentences — he wants to be in motion. End with: "Ready to branch and start, or do you want to talk through anything first?"

THE PROJECT

Internal tool for ILC Studios (lighting consultancy). Produces conceptual and design budgets, manages projects, and runs a CRM pipeline. Live at `https://app.ilcstudios.com`, auto-deploys from `main` via AWS Amplify.

Stack: Next.js 14 App Router, Tailwind CSS, Prisma 5.22 (SQLite locally / PostgreSQL in prod), JWT auth, Amplify SSR.

THE TEAM

Name	Role
Mike Kershner	Founder and lead dev. Pushes directly to main.
Sean McCauley	Manny's reviewer. Merges all Manny's PRs.
Manny (you)	Part-time dev ~10 hr/week. Always works on manny/* branches.

MANNY'S MANDATORY WORKFLOW — EVERY SESSION

Step	Command / Action
1	git pull (always first — never skip)
2	git checkout -b manny/<short-description>
3	Edit code, test with npm run dev:3000, iterate
4	git commit, then git push (pre-push hooks run ~25 sec — never use --no-verify)
5	gh pr create, then ping Sean to review. Never push to main directly.

PRE-PUSH HOOKS (NEVER BYPASS)

Six checks run automatically on every push: dynamic-pages, postgres-migrations, schema-sync, `npm test` (38 scenarios + 112K fuzz), check-secrets, push-size. If one fails, fix the underlying issue — **never use** `--no-verify`.

CRITICAL RULES

Rule	Detail
force-dynamic on every page	Every file under src/app/(app)/ must export: export const dynamic = 'force-dynamic' Without it the page cra
schema.prisma stays sqlite	provider = "sqlite" must never be changed — Amplify patches it to postgresql at build time.
Leave the budget engine alone	src/lib/calculations.js is validated. Do not touch unless a task explicitly says to.
Auth on every API route	Every route handler starts with: const { user, error } = await requireAuth(request)

LOCAL DEV LOGIN

```
npm run dev:3000

Email:    admin@ilcstudios.com
Password: ILCadmin2024!
```

CURRENT STATE (as of 2026-07-08)

Tasks 1–13 from the last three sprints all merged to main. Now on Tasks 1–3 in this document — one backend security sprint and two CRM enhancements. No schema changes, no engine changes.

KEY FILES FOR THIS SPRINT

File	Task
src/app/api/anthropic-models/route.js	Task 1 — fix 1: add auth
src/app/api/cron/client-review-check/route.js	Task 1 — fix 2: invert auth guard
src/app/api/crm/notes/route.js	Task 1 — fix 3: empty body validation
src/app/api/projects/[id]/meeting-notes/[noteId]/route.js	Task 1 — fix 4: attendees serialization
src/app/(app)/crm/CRMClient.jsx	Tasks 2 and 3 — list view + stats bar

Tasks 1–3 overview

Task 1 is four small backend fixes — good warmup, ~45 min total. Tasks 2 and 3 are both in CRMClient.jsx. Work in order: 1 → 2 → 3.

Task	Branch	File(s)	Effort
1 — Security hygiene (4 fixes)	manny/security-hygiene	4 API route files	~45 min
2 — CRM list view polish	manny/crm-listview-polish	CRMClient.jsx	~1 hr
3 — Win rate + weighted pipeline	manny/crm-stats-winrate	CRMClient.jsx	~45 min

Task 1 — Security hygiene: four micro-fixes

Branch: `manny/security-hygiene` | 4 API route files | ~45 min

A 360° audit found four small but real bugs. Each fix is 1–3 lines. Read each file before touching it. Apply all four, then test the app.

Fix 1 — add auth to anthropic-models (CRITICAL)

File: `src/app/api/anthropic-models/route.js`

The GET handler has **zero auth** — any unauthenticated caller on the internet can hit this endpoint. Add `requireAuth` the same way every other route does it:

```
// Add at top of file (with other imports):
import { requireAuth } from '@/lib/auth';

// Change function signature and add auth check:
export async function GET(request) {
  const { user, error } = await requireAuth(request);
  if (error) return error;

  const apiKey = getAnthropicKey()?.trim();
  // ... rest of handler unchanged
}
```

Fix 2 — invert auth guard in client-review-check cron

File: `src/app/api/cron/client-review-check/route.js`

Current code only checks auth *when* `CRON_SECRET` is set. If the env var is ever missing, any caller can trigger alert creation across all clients. Replace the current conditional block (lines 20–26) with:

```
// REMOVE this:
const cronSecret = process.env.CRON_SECRET;
if (cronSecret) {
  const authHeader = request.headers.get('authorization');
  if (authHeader !== `Bearer ${cronSecret}`) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
  }
}

// REPLACE with:
const cronSecret = process.env.CRON_SECRET;
const authHeader = request.headers.get('authorization');
if (!cronSecret || authHeader !== `Bearer ${cronSecret}`) {
  return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
}
```

Fix 3 — reject empty note body in notes POST

File: `src/app/api/crm/notes/route.js`

The POST handler accepts an empty `noteBody` string — a mis-click on '+ Note' creates a real DB row with a blank body. Find the destructuring block (around line 66) and add one validation check right after it:

```
// After the const { title, noteBody, ... } = body; block:
if (!noteBody?.trim()) {
  return NextResponse.json(
    { error: 'Note body is required' },
    { status: 400 }
  );
}
```

Fix 4 — fix attendees serialization in meeting-notes PATCH

File: `src/app/api/projects/[id]/meeting-notes/[noteId]/route.js`

Line 28 stores `attendees` directly without checking if it's an array. If the client sends an array, Prisma stores the string `[object Object]` instead of valid JSON. The POST handler in the parent `meeting-notes/route.js` already has the correct guard — apply the same pattern here:

```
// Change line 28 from:
if (attendees !== undefined) data.attendees = attendees || '[]';

// To:
if (attendees !== undefined)
  data.attendees = typeof attendees === 'string'
    ? attendees
    : JSON.stringify(attendees || []);
```

Acceptance criteria

* All four files changed. The app runs without errors — `npm run dev:3000`, log in, open the CRM.
* Pre-push hooks pass. No --no-verify.

Task 2 — CRM list view polish

Branch: `manny/crm-listview-polish` | File: `src/app/(app)/crm/CRMClient.jsx` | ~1 hour

Two small gaps vs. the kanban view: (A) notes count badges only appear on kanban cards, not in list view rows; (B) there's no way to clear all filters at once.

Part A — notes count in list view

The `project._count.noteLinks` value is already fetched (added to the Prisma query in the last sprint). It's just not rendered in `ListView`.

Find the `ListView` function (~line 292). In the `<tbody>` rows, find the `<td>` that renders the project name (~line 356):

```
<td className="px-3 py-2.5">
  <p className="font-semibold text-slate-900 truncate max-w-[200px]">
    {project.name}
  </p>
</td>
```

Change it to:

```
<td className="px-3 py-2.5">
  <p className="font-semibold text-slate-900 truncate max-w-[200px]">
    {project.name}
  </p>
  {(project._count?.noteLinks ?? 0) > 0 && (
    <span className="text-[10px] text-slate-400">
      {project._count.noteLinks}
      {project._count.noteLinks !== 1 ? ' notes' : ' note'}
    </span>
  )}
</td>
```

Part B — clear all filters button

The filter state has five dimensions: `filters.search`, `filters.status`, `filters.serviceType`, `filters.salesOwnerId`, and the separate `needsAttention` boolean. Resetting them all currently requires 4–5 clicks.

Find the filters toolbar JSX (search for `placeholder="Search projects…"`, ~line 551). After the sales owner `<select>` (~line 578), add:

```
{(filters.search || filters.status || filters.serviceType
  || filters.salesOwnerId || needsAttention) && (
  <button
    onClick={() => {
      setFilters({ search: '', status: '', serviceType: '', salesOwnerId: '' });
      setNeedsAttention(false);
    }}
    className="px-2.5 py-1.5 text-[10px] font-bold rounded-lg
               text-slate-400 hover:bg-slate-100 border border-slate-200"
  >
    Clear
```

```
  </button>
)}
```

The button only appears when at least one filter is active. Clicking it resets everything.

Acceptance criteria

* Notes count appears under project names in list view (same text as kanban badge).
* 'Clear' button appears in the filter bar whenever any filter is active; clicking it resets all five dimensions at once.
* No API changes, no schema changes. Pre-push hooks pass.

Task 3 — Win rate + weighted pipeline in stats bar

Branch: `manny/crm-stats-winrate` | File: `src/app/(app)/crm/CRMClient.jsx` | ~45 min

The pipeline stats bar already shows 6 chips. Two high-value metrics are missing: **win rate** (what % of closed deals do we win?) and **weighted pipeline** (open revenue × probability — a realistic forecast vs. raw open pipeline). Both are pure calculations from data already in state.

Where to make the change

Find the `PipelineSummary` function (~line 261). It currently computes 6 stats. Here is the current state of the function body up to the stats array:

```
function PipelineSummary({ projects, pipeline }) {
  const sf       = statusFieldFor(pipeline);
  const open     = projects.filter(p => p[sf] === 'OPEN');
  const won      = projects.filter(p => p[sf] === 'WON' && daysSince(p.updatedAt) <= 30);
  const lost     = projects.filter(p => p[sf] === 'LOST');
  const highProb = open.filter(p => (p.pipelineProbability ?? 0) >= 80);
  const stale    = open.filter(p => daysSince(p.updatedAt) >= 30);
  const sum = arr => arr.reduce((s, p) => s + (revenueFor(p, pipeline) || 0), 0);

  const stats = [
    { label: 'Open Pipeline',    value: fmtMoney(sum(open))     || '—', color: '...' },
    { label: 'High Prob (80%+)', value: fmtMoney(sum(highProb)) || '—', color: '...' },
    { label: 'Won (30 days)',    value: fmtMoney(sum(won))      || '—', color: '...' },
    { label: 'Lost',             value: lost.length,                    color: '...' },
    { label: 'Open Projects',    value: open.length,                    color: '...' },
    { label: 'Needs Attention',  value: stale.length,                   color: '...' },
  ];
```

What to add

After the `const stale = ...` line, add two new derived values:

```
const wonAll = projects.filter(p => p[sf] === 'WON');  // all-time for win rate
const winRate = wonAll.length + lost.length > 0
  ? `${Math.round(wonAll.length / (wonAll.length + lost.length) * 100)}%`
  : '\u2014';  // em dash when no closed deals
const weightedTotal = open.reduce(
  (s, p) => s + (revenueFor(p, pipeline) || 0) * (p.pipelineProbability ?? 0) / 100,
  0
);
```

Then add two entries to the `stats` array. Insert 'Weighted Pipeline' after 'High Prob (80%+)' and 'Win Rate' after 'Won (30 days)'. Final order will be 8 chips:

```
{ label: 'Open Pipeline',    value: fmtMoney(sum(open))           || '—', color: 'text-slate-900'  },
{ label: 'High Prob (80%+)', value: fmtMoney(sum(highProb))       || '—', color: 'text-orange-600' },
{ label: 'Weighted Pipeline',value: fmtMoney(Math.round(weightedTotal)) || '—',
                                                                          color: 'text-blue-700'   },
{ label: 'Won (30 days)',    value: fmtMoney(sum(won))            || '—', color: 'text-green-700'  },
{ label: 'Win Rate',         value: winRate,
  color: wonAll.length >= lost.length ? 'text-green-700' : 'text-red-600' },
{ label: 'Lost',             value: lost.length,                          color: 'text-red-600'    },
{ label: 'Open Projects',    value: open.length,                          color: 'text-slate-700'  },
```

```
{ label: 'Needs Attention',  value: stale.length,
  color: stale.length > 0 ? 'text-amber-600' : 'text-slate-400' },
```

Note: the chip rendering JSX below the stats array uses `flex-1` on each chip so they scale automatically — no layout changes needed for 8 chips.

Acceptance criteria

* Stats bar shows 8 chips including 'Weighted Pipeline' ($ value) and 'Win Rate' (%).
* Win Rate is green when wins ≥ losses, red otherwise. Shows '—' when no closed deals.
* Both new chips update when switching between Procurement and Design tabs.
* No other components changed. Pre-push hooks pass.

Operating reminder (for Claude): Tell Manny what to do — not how to do it technically. He approves proposed changes; Claude handles git, tests, and pushes. Manny never pushes to main — always via a manny/* branch and PR to Sean. If pre-push hooks fail, diagnose and fix. Never --no-verify.